Institute of Technology of Cambodia
Faculty of Electrical Engineering
Génie Informatique et Communication

GICI4OS · OPERATING SYSTEMS · LAB 5

Threads, Workers and Signals

Lab 5 guide: start workers, wait for each one to finish, and tell completion apart from protecting shared data.



Heng Rathpisey · Department GIC · Academic year 2025–2026

Learning objectives

1. Create POSIX threads, and check the return codes of the API.
2. Join each worker before using its result, and explain shared versus separate address spaces.
3. Use a bounded run to find missing completion synchronization.

THIS LAB

Lab 5

First the concepts, with code. Then the lab on the server. Process versus thread memory, kernel workers and signals are optional extensions, so they come last.

DISCUSSION

Two workers start, and main returns at once. What happens to them?

Do both finish their work and print? Does main know when they are done?

CONCEPTS FIRST

Processes, threads and joining

Three ideas: what a thread shares, how you wait for it, and what waiting does not do.

A process and a thread

	Process	Thread
What it is	A running program with its own resources	A line of execution inside a process
Address space	Its own: memory is private	Shared with the other threads of the process
Created by	`fork()`	`pthread_create()`
Waited for with	`waitpid()`	`pthread_join()`
Cost to create	Higher	Lower
Risk	Needs a channel to share data	Threads can overwrite each other's data

After fork, parent and child have separate copies of memory. Threads of one process see the same variables.

What threads share, and what they do not

SHARED

Shared by all threads

Code, global variables, the heap, open files. A change made by one thread is visible to the others.

PRIVATE

Private to each thread

Its own stack with local variables, its registers and its place in the code.

TRADE-OFF

Why it matters

Sharing makes communication cheap, and it also lets two threads change one value at the same time.

Create and join

Function	What it does	Error reporting
`pthread_create`	Starts a new thread that runs a function	Returns an error number, 0 on success
`pthread_join`	Waits until one particular thread has finished	Returns an error number, 0 on success
`strerror(rc)`	Turns an error number into text	Use it instead of errno here

These functions return the error number directly. They do not set errno. Check every return value. Compile with -pthread.

Completion is not protection

COMPLETION

Joining answers: is it finished?

pthread_join waits for one worker to finish. After the join, main may read what that worker wrote.

MUTUAL EXCLUSION

A mutex answers: who goes now?

A lock lets only one thread at a time touch a shared value. A join does not do this.

SCOPE

This lab

Each worker writes its own field, so no lock is needed. Joining is the core. Locks come in a later lab.

LAB 5

Workers that finish before main reports

Alex is making a TechCorp worker program concurrent. Each worker must finish before main reports its result.

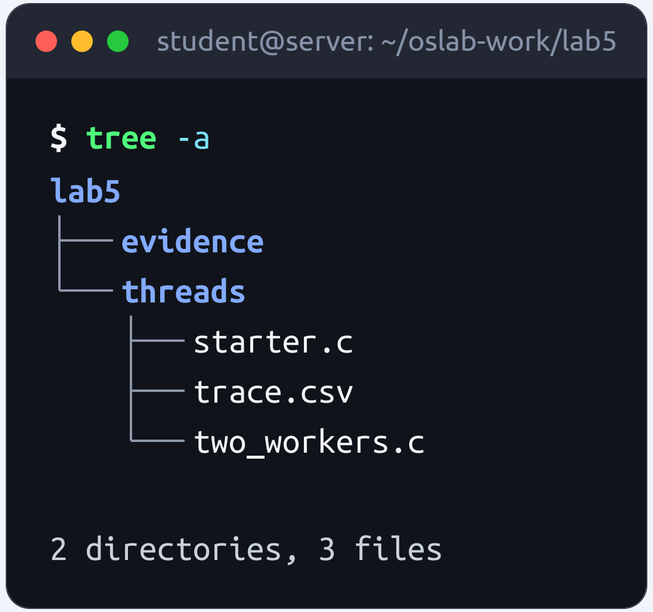
The scenario

SCENARIO

Alex at TechCorp

Make a worker program concurrent. Each worker must finish before main reports its result. Compare thread completion with protecting shared mutable memory.

Your worker ids and step counts are your own. This guide shows example values.



Inside the 120 minutes

Minutes	What happens
0 to 10	Setup
10 to 25	Guided: one worker
25 to 35	Prediction
35 to 55	Core 1: finish the two-worker program
55 to 70	Core 2: test and change

Minutes	What happens
70 to 85	Instructor demonstration, Plus and Challenge
85 to 100	Live checkpoint
100 to 110	Debrief
110 to 120	Submit

Six helper commands, and why you use them

Command	What it does
`oslab values lab5`	Shows your worker ids and step counts. They are already written in your program
`oslab predict lab5`	Saves your prediction once, before the experiment
`oslab hint lab5 1`	A hint, levels 1 to 3. Hints cost no points
`oslab check lab5`	Compiles and runs your program on test cases and shows your milestones
`oslab checkpoint lab5`	Opens when the instructor says so, near the end
`oslab debrief lab5`	After the lab: you give the right answers to your prediction questions

Before the lab, run oslab doctor and oslab prelab lab5. It takes about ten minutes.

How you work

Core, Plus, Challenge

Core is for everyone. Plus and Challenge are for students who finish early. Full marks need only Core and the checkpoint.

AI tools

Allowed in Core, Plus and Challenge, so test what they tell you. Not allowed in the prediction and the checkpoint.

A program with two gaps

In Core 1 you receive a program with two gaps. You decide where each join goes and what main may read before it.

Never signal a kernel worker or another user's process. A wrong prediction costs nothing.

Setup: start the lab

student@server: ~

```
$ export OSLAB_WORKSPACE="${OSLAB_WORKSPACE:-$HOME/oslab-work}"
$ oslab start lab5
$ cd "$OSLAB_WORKSPACE/lab5"
$ mkdir -p evidence
$ oslab values lab5
```

- The first line sets OSLAB_WORKSPACE to ~/oslab-work, unless it is already set. Type it as it is.
- oslab start lab5 makes the lab folder.
- cd takes you into it, and mkdir -p makes the evidence folder.
- oslab values lab5 prints your own numbers.

Your values: four numbers that are only yours

The last command prints them

example: your values will be different

```
$ oslab values lab5
Your personal values for lab5 (your-account):
  id_a     = 3
  id_b     = 9
  steps_a  = 5
  steps_b  = 2
```

What they mean

- Every step of a worker adds the worker's id to its result.
- In the example, worker A ends with 3 x 5 = 15 and worker B with 9 x 2 = 18.
- The numbers are already written in the line struct job a = ... of your program.

Your folder

student@server: ~/oslab-work/lab5

```
$ tree -a -L 3
lab5
├── evidence
└── threads
    ├── starter.c
    ├── trace.csv
    └── two_workers.c

2 directories, 3 files
```

threads/two_workers.c is your program. It has two gaps, marked TODO, that you fill.

starter.c is a short two-worker reminder. trace.csv is an illustration of two workers reading and writing one shared value.

Check the compiler

student@server: ~/oslab-work/lab5

```
$ command -v gcc
$ command -v timeout
$ oslab check lab5
```

command -v prints the path of a program, and nothing if it does not exist. timeout stops a command after a time limit. Every run in this lab uses it.

At the start every line of oslab check says TRY, because the program has no joins and no report line yet.

CAREFUL

Do not install

Do not install packages on the shared server. If gcc is missing, tell the instructor: the lab has a trace fallback, and you must report that the behaviour was not tested.

GUIDED

One worker

Ten to twenty-five minutes, with the instructor. A complete example with one worker.

Step 1: the one-worker program

threads/one_worker.c

```c
#include <pthread.h>
#include <stdio.h>
#include <string.h>
static void *worker(void *arg) {
    int *result = arg;
    *result = 6 * 7;
    return NULL;
}
int main(void) {
    int result = 0;
    pthread_t thread;
    int rc = pthread_create(&thread, NULL, worker, &result);
    if (rc) { fprintf(stderr, "create: %s\n", strerror(rc)); return 1; }
    rc = pthread_join(thread, NULL);
    if (rc) { fprintf(stderr, "join: %s\n", strerror(rc)); return 1; }
    printf("result=%d\n", result);
    return 0;
}
```

Create threads/one_worker.c with cat > threads/one_worker.c <<'C', paste the lines, and end with a line that holds only C.

1. The worker writes result.
2. Main waits with pthread_join.
3. Main reads result after the join.

The worker gets a pointer to result, so both threads use the same memory.

Step 1: compile and run

student@server: ~/oslab-work/lab5

```
$ gcc -Wall -Wextra -pthread threads/one_worker.c -o threads/one_worker
$ timeout 5 ./threads/one_worker
result=42
```

-Wall -Wextra turn on warnings. -pthread links the thread library. -o names the program.

pthread_create starts the worker and pthread_join waits for that one worker. Both return an error code, where 0 means success, so the program tests rc.

Step 2: mark three lines

TRY

Mark them on your copy

The line where the worker writes result. The line where main waits. The line where main reads it.

THINK

Observe

Would creating a thread alone tell main when the result is ready? Would a join also stop two workers from racing on one shared counter?

REMEMBER

Compare with fork

Threads share the address space, so main sees what the worker wrote. After fork, each process would have its own private copy.

PREDICT

Guess first, then try

Before you run the two-worker program. Ten minutes.

The prediction

student@server: ~/oslab-work/lab5

```
$ oslab predict lab5
```

It asks three questions and one sentence, with your own ids and step counts. Answer alone, without AI.

You compare it with the real result in the debrief.

PREDICTION

The rules

Do not run the two-worker program yet. The first answer is the one that counts. A wrong guess costs nothing.

CORE 1

Finish the two-worker program

Twenty minutes, from minute 35 to 55. Main must wait for both workers, check the return codes, and only then report.

Step 1: read your program, the worker

threads/two_workers.c (1 of 2)

```
struct job { int id; int steps; int result; };
static void *worker(void *arg) {
    struct job *job = arg;
    const struct timespec delay = {0, 20000000};
    for (int i = 0; i < job->steps; ++i) {
        job->result += job->id;
        nanosleep(&delay, NULL); /* teaching delay, not synchronization */
    }
    printf("worker %d finished\n", job->id);
    return NULL;
}
```

Read it with cat -n threads/two_workers.c. Find the two workers, the two pthread_create calls and the two TODO lines.

Each worker adds its own id to its own result field, once per step.

The 20 ms delay only makes an early exit by main easier to see. It is not a way to wait, and it guarantees no order.

Step 1: read your program, main

threads/two_workers.c (2 of 2)

```
int main(void) {
    struct job a = {3, 5, 0}, b = {9, 2, 0};
    pthread_t ta, tb;
    int rc = pthread_create(&ta, NULL, worker, &a);
    if (rc) { fprintf(stderr, "create A: %s\n", strerror(rc)); return 1; }
    rc = pthread_create(&tb, NULL, worker, &b);
    if (rc) {
        fprintf(stderr, "create B: %s\n", strerror(rc));
        pthread_join(ta, NULL);
        return 1;
    }
    /* TODO: join both workers and check each return code. */
    /* TODO: after both joins, print a.result and b.result. */
    puts("main finishing");
    return 0;
}
```

Main creates both workers. If the second create fails, it joins the first before it leaves.

The line with struct job holds your own numbers. The example shows ids 3 and 9, steps 5 and 2.

The two marked TODO lines are your work.

Step 2: run it as it is

student@server: ~/oslab-work/lab5

```
$ gcc -Wall -Wextra -pthread threads/two_workers.c -o threads/two_workers
$ timeout 5 ./threads/two_workers
```

Main finishes without waiting. The workers may not have finished, or may not have printed.

TRY

Think

Does main finishing mean the workers finished?

Step 3: your decision

Replace the two TODO lines

1. Join each worker, with the thread variable of that worker, and test every return code like in the one-worker example.
2. After both joins, print one line: A result=NUMBER B result=NUMBER, using a.result and b.result.
3. Decide yourself which lines come before main finishing.

YOUR TURN

Open it with nano

nano threads/two_workers.c. With the example numbers the line reads A result=15 B result=18. Yours are different.

WHY

Think

Why must the report line come after the joins and not before them?

Step 4: compile again and run

student@server: ~/oslab-work/lab5

```
$ gcc -Wall -Wextra -pthread threads/two_workers.c -o threads/two_workers
$ timeout 5 ./threads/two_workers
```

Read every warning. The check wants none.

TRY

Think

Why does the order of the two worker finished lines not matter, while their position before the report line does?

Step 5: a different design, the trace

The trace shows two workers that read and then write one shared value.

student@server: ~/oslab-work/lab5

```
$ cat threads/trace.csv
```

THINK

Think

Joining them after their work would not stop the overlap in that design. What could go wrong?

Complete when, and hints

DONE

Complete when

Both workers complete before main reports. Every creation and join return code is tested. The report line shows your own numbers.

Hints: oslab hint lab5 1, 2, 3

1. Match one join to each thread that was created.
2. Compare the two thread variables you pass to the joins.
3. AI is allowed here. Test every suggestion.

Test and change

Fifteen minutes, from minute 55 to 70. Show that the program is right for other numbers too.

Step 1: save a normal run

student@server: ~/oslab-work/lab5

```
$ gcc -Wall -Wextra -pthread threads/two_workers.c -o threads/two_workers
$ timeout 5 ./threads/two_workers > evidence/normal.txt 2>&1
$ rc=$?; printf 'exit=%s\n' "$rc" >> evidence/normal.txt
$ cat evidence/normal.txt
```

The file holds the program output and the exit status. Exit status 124 would mean that timeout stopped the program. That is not a success.

$? is the exit status of the last command.

Step 2: a changed case, in a copy

student@server: ~/oslab-work/lab5

```
$ cp -- threads/two_workers.c threads/changed.c
$ nano threads/changed.c
$ gcc -Wall -Wextra -pthread threads/changed.c -o threads/changed
$ timeout 5 ./threads/changed > evidence/changed.txt 2>&1
$ rc=$?; printf 'exit=%s\n' "$rc" >> evidence/changed.txt
$ cat evidence/changed.txt
```

Work in a copy, so your own program keeps your own numbers. In changed.c give worker A 0 steps and worker B 1 step, and keep both ids.

TRY

Predict first

What will A and B print? A worker with zero steps still needs a join.

What each run shows

Run	Expected	What it tests
Normal	A result=id_a x steps_a, B result=id_b x steps_b, exit=0	Creation, joins, results read after completion
Changed: A 0 steps, B 1 step	A result=0, B result=id_b	That the result follows the steps, and a fast worker needs a join too
Exit status 124	Not a success	That the time limit expired

Several good runs do not prove that there is no race: a run can pass by luck. Without -pthread the build can fail.

Steps 3 and 4: check, then explain

The four Core milestones must show PASS: the program builds, both joins, the return codes, and the changed case. oslab check also runs a changed version of your program.

student@server: ~/oslab-work/lab5

```
$ oslab check lab5
```

REPORT

Write down for your report

What makes it safe for main to read a.result after the join? Why is the delay not a lock? What would change if both workers updated one shared field?

Plus: remove one join

```
student@server: ~/oslab-work/lab5

$ cp -- threads/two_workers.c threads/nojoin.c
```

In the copy, delete only the join of worker B. Run it three times, and write what main printed for B each time.

PLUS

Answer in evidence/plus.txt

When can main print B's final number? When can it print less?

First watch the instructor remove one join and run the program several times. The output is sometimes right, which is why a passing test does not prove the program. Plus and Challenge are not needed for full marks.

Challenge: read the trace

Two workers each read one shared value, which starts at 0, add 1, and write it back.

student@server: ~/oslab-work/lab5

```
$ cat threads/trace.csv
```

CHALLENGE

Answer in evidence/plus.txt

In the order of the file, what is the final value? Is it 2? Which line of the trace shows the problem?

LIVE CHECKPOINT

Show what you can do alone

Fifteen minutes near the end of the lab. Worth 3 of the 10 points.

Why is there a checkpoint?



It shows what you can do alone

For most of the lab you may use hints and AI tools. The checkpoint is the one moment without them.



Your numbers are new

The questions use new numbers, made for you at that moment. An answer copied from a neighbour does not fit.



The instructor sees who needs help

Your answers arrive at once, so a common mistake can be explained before the next lab.



It is fair

Everyone gets the same kind of task, with their own numbers.

What happens, step by step

1. The instructor opens it
Close your AI tools

→

2. oslab checkpoint lab5
Answer the four questions

→

3. The file task
Copy your program, new step counts

→

4. oslab check lab5
A fifth milestone must pass

Before it opens

The command says that the checkpoint is not open yet. Wait for the instructor.

While you work

Use your terminal and what you learned in this lab. No AI tools, and no help from a neighbour.

When you finish

oslab check lab5 shows whether the fifth milestone, checkpoint, passes. You can fix your copy and check again.

What the checkpoint asks

Part	What you do
Four questions	They use new numbers that were made for you at this moment. Answer them in the terminal.
The file task	Copy your program under a new name, give the workers the new step counts the command prints, run it, and save the output in checkpoint.txt.
The check	Run oslab check lab5. The fifth milestone, checkpoint, must show PASS.

Your own two_workers.c stays as it is. The task uses a copy.

DEBRIEF AND SUBMIT

Look back, then hand in

Ten minutes of debrief, ten to submit.

Debrief: look at your prediction again

student@server: ~/oslab-work/lab5

```
$ cat "$OSLAB_WORKSPACE/.records/lab5-predict.json"
$ oslab debrief lab5
```

oslab saved your prediction in a hidden folder, .records. It stays as it was, so the comparison shows which answers you confirmed and which you corrected.

DISCUSS

Class discussion

A program printed the right numbers three times. What does that prove?

Submit 1: your repository and the copy

student@server: ~

```
$ SUBMISSION_REPO="$HOME/os-gic-YOUR_ID/os-lab-YOUR_ID"
$ mkdir -p "$SUBMISSION_REPO/lab5/evidence"
$ cd "$OSLAB_WORKSPACE/lab5"
$ cp -- threads/two_workers.c "$SUBMISSION_REPO/lab5/"
$ cp -- evidence/normal.txt evidence/changed.txt "$SUBMISSION_REPO/lab5/evidence/"
```

Replace YOUR_ID with your own ID. The orange text is the part you change. Use the absolute path of your existing course repository, never a neighbour's.

Example: with the ID 0123, the first line becomes SUBMISSION_REPO="$HOME/os-gic-0123/os-lab-0123", and echo "$SUBMISSION_REPO" prints /home/student/os-gic-0123/os-lab-0123.

Copy the source, not the compiled program. If you did Plus or Challenge, copy evidence/plus.txt the same way. The course website checks that these exact names are there.

Submit 2: the final tree and Git

student@server: ~/os-gic-YOUR_ID/os-lab-YOUR_ID

```
$ tree
lab5
├── evidence
│   ├── changed.txt
│   └── normal.txt
├── README.md
└── two_workers.c

1 directory, 4 files
```

1. Write README.md from the lab's template, in your own words.
2. The prediction must have been written before you ran anything. Copying it into the README only records it.
3. git status --short, then add only your lab files, then commit and push.
4. No passwords, personal shell files, generated binaries or disk images.

Submit 3: commit and push

student@server: ~

```
$ cd ~/os-gic-YOUR_ID
$ git pull
$ git add os-lab-YOUR_ID/lab5
$ git commit -m "Lab 5"
$ git push
```

git pull comes first, in case you pushed from your own computer since.

Replace YOUR_ID with your own ID, as before.

How it is marked: 10 points

Evidence for the lab objectives	Points
Program: oslab check lab5 passes the four Core milestones (builds, both joins, return codes, changed case)	2
Tests: normal.txt and changed.txt show the runs, and the README explains thread memory, completion and mutual exclusion	2
Prediction saved in time, and corrected with oslab debrief lab5	2
Live checkpoint: the four answers and your copy running with the new step counts	3
Complete submission: the required files in your repository, on time	1

Every row is marked from your oslab records and the files in your repository, so run oslab check lab5 until it shows what you expect. Your prediction earns its first point for being made, not for being right. Plus and Challenge are not needed for full marks.

Common mistakes

Mistake	What you see	Fix
No -pthread in the gcc line	A build or link error	Add -pthread
Reading a result before the join	An old value, sometimes right	Move the reads after both joins
Joining only one worker	A message that sometimes goes missing	Join every thread that was created
Trusting the delay as a lock	It works until the timing changes	Join, and use a mutex for shared writes
Changing two_workers.c for the changed case	Your evidence no longer has your numbers	Change the copy, changed.c

BEYOND THE CORE

Fork, kernel workers and signals

Optional extensions. Each uses a bounded run and your own processes.

Lab 5

Fork copies memory, threads share it

a small fork() program

```
$ ./fork_demo
child: value=9 pid=4242
parent: value=1 pid=4241
```

Set an integer to 1. Change it to 9 only in the child. The parent waits with waitpid, then prints.

The parent still sees 1: after fork the two processes have separate memory. Threads would see 9.

Check fork and waitpid for errors, flush output before fork, and run under timeout 5.

Looking at threads from outside

student@server: ~/oslab-work/lab5

```
$ ./threads/two_workers & pid=$!
$ ps -L -p "$pid" -o pid,lwp,nlwp,comm
$ wait "$pid"
$ ps -eo pid,ppid,comm | head -n 20
```

ps -L lists each thread: LWP is the thread's ID, and NLWP counts the threads of the process.

Raise the worker delay to about two seconds per step, so you have time to look.

Kernel workers may be hidden inside a container or WSL. Not seeing them does not prove they are absent. Observe them only; never signal them.

A signal handler that does almost nothing

signal_demo.c

```
static volatile sig_atomic_t stop;
static void on_term(int sig) {
    (void)sig;
    stop = 1;  /* only set a flag */
}
/* in main: register it, then loop */
struct sigaction sa = {0};
sa.sa_handler = on_term;
sigaction(SIGTERM, &sa, NULL);
while (!stop) { /* work */ }
/* print and clean up here, after the loop */
```

The handler only sets a flag. Printing and cleanup happen in the normal loop, not inside the handler.

Keep a ten-second limit. Start it, note its own PID, and send kill -TERM to that PID only.

A join waits for completion. A mutex protects shared data. A signal flag is neither.

Key takeaways

01

Thread or process

Threads share memory in one process. Forked processes get separate copies.

02

Create, then join

Check every return code, and join each thread you created.

03

Completion is not protection

A join says it finished. A mutex says who may write now.

04

Bound every run

timeout 5 keeps a mistake from running forever. Exit 124 means it ran out of time.

Next session

COMING UP

Lab 6: users, groups and file permissions

Who may read, write or run a file, and how Linux decides.

SOURCE

Reading

Silberschatz, Galvin and Gagne, *Operating System Concepts*, 10th edition, chapter 4, Threads and Concurrency.